Deep Learning Practice Set

(All notations follow those used in the lecture slides)

***Learning rate is given as η**

Section A: Multi Layer Perceptron (MLP)

1. For a neuron with input $x = [1, 2]$, weights $w = [0.5, -1]$, and bias $b = 1$, compute the output before activation.
2. Using ReLU activation, compute the output for $z = -3$.
3. Using sigmoid activation, compute the output for $z = 0$.
4. Compute the output of a neuron with:
   - Inputs: $[2, -1, 3]$
   - Weights: $[0.2, 0.5, -0.3]$
   - Bias: 0.1
5. Compute the derivative of sigmoid at $z = 0$.
6. If a layer has 5 neurons and each neuron receives 3 inputs, compute total number of weights (ignore bias).
7. Compute total parameters (including bias) for a layer with:
   - Input size = 4
   - Output neurons = 3
8. Compute the output of tanh at $z = 0$.
9. Given:
   - Input: $x = [1, 2]$
   - Weights:
   $$W = \begin{bmatrix} 0.1 & 0.2 \\ 0.3 & 0.4 \end{bmatrix}$$
   - Bias: $b = [0.5, 0.5]$

   Compute:
   - Linear output
   - Output after sigmoid activation

10. Given:

    - Input: $x = [1, 0]$

    Layer 1:

    $$W_1 = \begin{bmatrix} 0.5 & -0.5 \\ 0.3 & 0.8 \end{bmatrix}, \quad b_1 = [0.1, 0.2]$$

    Layer 2:

    $$W_2 = \begin{bmatrix} 0.7 & 0.6 \end{bmatrix}, \quad b_2 = [0.3]$$

    Use ReLU in hidden layer and sigmoid in output. Compute final output.

11. An MLP has:

    - Input layer: 10 neurons
    - Hidden layer: 6 neurons
    - Output layer: 2 neurons

    Compute total number of parameters (including bias).

12. Given:

    - Input: $x = 2$
    - Weight: $w = 0.5$
    - Bias: $b = 0$
    - Target: $y = 1$

    Activation: Linear.

    Compute:

    - Output
    - Loss using Mean Squared Error
    - Gradient of loss w.r.t weight

13. Given:

    $$z = wx + b$$

    Activation: Sigmoid, Loss:

    $$L = \frac{1}{2}(y - \hat{y})^2$$

    Derive:

    $$\frac{\partial L}{\partial w}$$

14. Given:

    - Input vector size: 1×3
    - Weight matrix size: 3×4

    Compute:

    - Output size

- General expression for output

15. Given:

    - Weight = 0.8
    - Gradient = 0.2
    - Learning rate = 0.1

    Compute updated weight using gradient descent.

16. For the network:

$$x \to z = wx \to a = \sigma(z) \to L$$

    Derive:

$$\frac{\partial L}{\partial w}$$

17. Given:

$$\sigma'(z) = \sigma(z)(1 - \sigma(z))$$

    Compute derivative at:

    - $z = 10$
    - $z = -10$

18. Network:

    - Input = 8
    - Hidden1 = 16
    - Hidden2 = 8
    - Output = 4

    Compute total parameters (including bias).

Section B: Deep Learning Optimizers and Training Techniques

1. Given for **SGD**:

$$\theta = [2, -1], \quad \mathbf{g} = [0.5, -0.2], \quad \eta = 0.1$$

    Compute updated weights using SGD.

2. Given for **Adagrad**:

$$\theta_{t-1} = [1.0, -2.0], \quad \mathbf{g}_t = [0.5, -0.5], \quad \eta = 0.1$$

    Adagrad accumulator update:

$$\mathbf{r}_t = \mathbf{r}_{t-1} + \mathbf{g}_t \odot \mathbf{g}_t$$

    Weight update:

$$\theta_t = \theta_{t-1} - \frac{\eta}{\delta + \sqrt{\mathbf{r}_t}} \odot \mathbf{g}_t$$

    If:

$$\mathbf{r}_{t-1} = [0, 0], \quad \delta = 10^{-8}$$

    Compute:

- $\mathbf{r}_t$
- Updated weights θ_t

3. Given **Momentum** update:

$$\mathbf{u}_t = 0.9\mathbf{u}_{t-1} - 0.1\mathbf{g}_t$$

If:

$$\mathbf{u}_{t-1} = [0.2, -0.1], \quad \mathbf{g}_t = [0.5, 0.3]$$

Compute $\mathbf{u}_t$.

4. For **RMSProp**:

$$\mathbf{r}_t = 0.9\mathbf{r}_{t-1} + 0.1\mathbf{g}_t \odot \mathbf{g}_t$$

Given:

$$\mathbf{r}_{t-1} = [0.4, 0.2], \quad \mathbf{g}_t = [0.5, -0.5]$$

Compute $\mathbf{r}_t$.

5. Compute standardization:

$$\mathbf{z} = \frac{\mathbf{x} - \boldsymbol{\mu}}{\boldsymbol{\sigma}}$$

for:

$$\mathbf{x} = [10, 12], \quad \boldsymbol{\mu} = [8, 10], \quad \boldsymbol{\sigma} = [2, 1]$$

6. Compute min-max normalization:

$$\mathbf{x}' = \frac{\mathbf{x} - \mathbf{x}_{min}}{\mathbf{x}_{max} - \mathbf{x}_{min}}$$

for:

$$\mathbf{x} = [6, 8], \quad \mathbf{x}_{min} = [2, 4], \quad \mathbf{x}_{max} = [10, 12]$$

7. Given learning rate scheduler after **t**-epochs as:

$$\eta_t = \frac{\eta_0}{1 + kt}$$

Compute η_t for:

$$\eta_0 = 0.1,\ k = 0.01,\ t = 20$$

8. **SGD (Vector Case – Multiple Steps)**

Given:

$$\theta_0 = [1.0, -1.0], \quad \eta = 0.1$$

Gradients:

$$\mathbf{g}_1 = [0.5, -0.3], \quad \mathbf{g}_2 = [0.2, 0.4]$$

Compute:

- θ_1
- θ_2

9. **Momentum (Vector Update)**

   Given:

$$\theta_0 = [1.0, 2.0], \quad \eta = 0.1, \quad \alpha = 0.9$$
$$\mathbf{u}_0 = [0, 0]$$

   Gradients:

$$\mathbf{g}_1 = [0.5, -0.2], \quad \mathbf{g}_2 = [0.3, 0.1]$$

   Compute:

   - $\mathbf{u}_1, \theta_1$
   - $\mathbf{u}_2, \theta_2$

10. **Nesterov Momentum (Vector Case)**

    Given:

$$\theta_0 = [1.0, -1.0], \quad \eta = 0.1, \quad \alpha = 0.9$$
$$\mathbf{u}_0 = [0, 0]$$

    Gradient at lookahead point:

$$\mathbf{g}_1 = [0.6, -0.4]$$

    Compute updated weight θ_1.

11. **Adagrad**

    Given:

$$\theta_0 = [1.0, -1.0], \quad \eta = 0.1$$
$$\mathbf{r}_0 = [0, 0], \quad \delta = 10^{-8}$$

    Gradients:

$$\mathbf{g}_1 = [0.5, -0.2], \quad \mathbf{g}_2 = [0.4, 0.3]$$

    Compute:

    - $\mathbf{r}_1, \theta_1$
    - $\mathbf{r}_2, \theta_2$

12. **RMSProp (Vector Update)**

    Given:

$$\theta_0 = [2.0, 1.0], \quad \eta = 0.01$$
$$\rho = 0.9, \quad \mathbf{r}_0 = [0, 0]$$
$$\mathbf{g}_1 = [0.5, -0.2]$$

    Compute:

    - $\mathbf{r}_1$
    - Updated θ_1

13. **Adam Optimizer (Vector Case)**

Given:

$$\theta_0 = [1.0, -1.0], \quad \eta = 0.01$$
$$\rho_1 = 0.9, \quad \rho_2 = 0.999$$
$$\mathbf{r}_0 = [0, 0], \quad \mathbf{s}_0 = [0, 0]$$
$$\mathbf{g}_1 = [0.5, -0.3]$$

Compute:

- $\mathbf{r}_1, \mathbf{s}_1$
- Bias-corrected $\hat{\mathbf{r}}_1, \hat{\mathbf{s}}_1$
- Updated θ_1

14. **Learning Rate Scheduler (after t epochs)**

Given:

$$\eta_t = \eta_0 \cdot \gamma^t$$

$$\eta_0 = 0.1, \quad \gamma = 0.9$$

Weights:

$$\theta_0 = [1, 2], \quad \mathbf{g}_1 = [0.5, 0.5]$$

Compute:

- Learning rate at $t = 2$
- Updated weights θ_1

15. **Xavier Initialization**

A weight matrix connects 64 input neurons to 32 output neurons.

Compute:

- Variance using Xavier initialization
- Standard deviation

16. **LeCun Initialization**

Given:

- Input neurons = 128
- Weight matrix size = 128×64

Compute:

- Variance
- Range (± 1 standard deviation)

17. **Batch Normalization**

    Given:

    $$\mathbf{x} = [2, 4, 6, 8]$$

    Compute:

    - Mean
    - Variance
    - Normalized values

    Then compute:

    $$y = \gamma x_{norm} + \beta$$

    for $\gamma = 2$, $\beta = 1$.

18. **Standardization vs Normalization**

    Given:

    $$\mathbf{x} = [5, 10, 15]$$

    Compute:

    - Standardized values
    - Min-max normalized values

Section C: Convolutional Neural Networks (CNN)

1. Compute the output size for:
    - Input = $280 \times 250 \times 3$, Kernel = 3×3, Stride = 1, Padding = 0
    - Compute the no. of parameters if the CNN needs to learn 8 such filters
2. Compute the output size for:
    - Input = 320×240, Kernel = 5×5, Stride = 1, Padding = 2
    - Compute the no. of parameters if the CNN needs to learn 16 such filters
3. After 2×2 max pooling (stride = 2), find the output volume of a $320 \times 240 \times 16$ feature volume.
4. Compute the output size:
    - Input = 64×64, Kernel = 7×7, Stride = 1, Padding = 3
    - Compute the no. of parameters if the CNN needs to learn 16 such filters
    - If the stride increases from 1 to 2, how do the output size and the number of parameters change? Justify numerically.
5. An input image of size 64×64 passes through:
    - Conv1: Kernel = 3×3, Stride = 1, Padding = 1
    - Pool1: 2×2, Stride = 2
    - Conv2: Kernel = 5×5, Stride = 1, Padding = 0

Compute the final output size.

6. A CNN layer has:

   - Input size $= 32 \times 32 \times 3$
   - Filters $= 20$
   - Kernel size $= 5 \times 5$

   Compute:

   - Total number of parameters
   - Output feature map size (Stride $= 1$, Padding $= 0$)

7. Perform convolution:

   Input:

   $$\begin{bmatrix} 2 & 1 & 0 & 2 \\ 1 & 3 & 1 & 0 \\ 0 & 2 & 2 & 1 \\ 1 & 0 & 1 & 3 \end{bmatrix}$$

   Kernel:

   $$\begin{bmatrix} 1 & 0 \\ -1 & 1 \end{bmatrix}$$

   Stride $= 1$, Padding $= 0$. Compute the output feature map.

8. For input size 32×32, compute output sizes:

   - Case 1: Kernel $= 3 \times 3$, Stride $= 1$, Padding $= 0$
   - Case 2: Kernel $= 3 \times 3$, Stride $= 2$, Padding $= 1$

9. Input has 3 channels. A convolution layer has:

   - 12 filters
   - Kernel size $= 3 \times 3$

   Compute total parameters (including bias).

10. Input $= 28 \times 28$ passes through:

    - Conv: 3×3, Stride $= 1$, Padding $= 0$
    - Max Pool: 2×2, Stride $= 2$
    - Conv: 3×3, Stride $= 1$, Padding $= 0$

    Find final output size.

11. A CNN produces 16 feature maps of size 8×8.

    - Find number of neurons after flattening
    - If fully connected layer has 100 neurons, compute total parameters

12. A CNN has:

- Layer 1: 3×3, stride 1
- Layer 2: 3×3, stride 1

Compute the receptive field.

13. Input: $128 \times 128 \times 3$

    Conv layer:

    - 32 filters
    - Kernel $= 3 \times 3$
    - Stride $= 1$
    - Padding $= 1$

    Find:

    - Output volume
    - Total parameters

14. Input: $32 \times 32 \times 3$

    Layers:

    - Conv1: 3×3, stride 1, padding 1, filters $= 8$
    - Pool1: 2×2, stride 2
    - Conv2: 3×3, stride 1, padding 1, filters $= 16$
    - Pool2: 2×2, stride 2

    Compute output size after each layer and final flattened size.

15. Consider an input of size $50 \times 50 \times 3$ passed through the following layers:

    - Conv1: 5×5, stride $= 1$, padding $= 2$, filters $= 6$
    - Pool1: 2×2, stride $= 2$
    - Conv2: 3×3, stride $= 1$, padding $= 1$, filters $= 16$

    Compute:

    - Output size after each layer
    - Total number of parameters in Conv1 and Conv2